



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Ayudantía 02: C# parte 3

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

22 de Agosto, 2025

Nullable Types

The null keyword

También existe el keyword `null` que representa una *null reference*.

```
2 Persona juan = null;  
3 if (juan == null)  
4     Console.WriteLine("Juan es null");
```

`null` es una referencia a la nada.

The null keyword

También existe el keyword `null` que representa una *null reference*.

```
2 Persona juan = null;
3 if (juan == null)
4     Console.WriteLine("Juan es null");
```

`null` es una referencia a la nada.

Cuando se crean arreglos de clases su valor por defecto es `null`.

```
2 Persona[] personas = new Persona[3];
3 personas[1] = new Persona("juan");
4 foreach (Persona p in personas)
5 {
6     if (p != null)
7         p.MostrarNombre();
8     else
9         Console.WriteLine("Persona no creada!");
10 }
```

The null keyword

Dado que `null` es, por definición, una referencia no se puede asignar `null` a variables que sean de tipos por valor.

```
2 int i = null;    // ERROR
3 Point p = null; // ERROR (Point es un struct)
```

The null keyword

Dado que `null` es, por definición, una referencia no se puede asignar `null` a variables que sean de tipos por valor.

```
2 int i = null;    // ERROR
3 Point p = null; // ERROR (Point es un struct)
```

Para poder asignar `null` a un tipo por valor hay que marcarlo como “*nullable*”

```
4 int? i = null;    // FUNCIONA
5 Point? p = null;  // FUNCIONA
```

Enumeraciones

Enumeraciones

Los enum mapean nombres a valores (de forma explícita o implícita).

Ejemplos (definición)

- `enum Color { Red, Blue, Yellow }`
- `enum Animal { Elephant = 5, Dog = 27, Cat = 2 }`
- `enum State { Normal = 1, Warning, Error }`

Enumeraciones

Los enum mapean nombres a valores (de forma explícita o implícita).

Ejemplos (definición)

- `enum Color { Red, Blue, Yellow }`
- `enum Animal { Elephant = 5, Dog = 27, Cat = 2 }`
- `enum State { Normal = 1, Warning, Error }`

Ejemplo (uso)

```
Color c = Color.Red;  
...  
if (c == Color.Yellow)  
    ...
```

Enumeraciones

Los `enums` son super útiles para definir atributos categóricos.

Enumeraciones

Los enums son super útiles para definir atributos categóricos.

```
2 class Estacion {
3     private List<Pasajeros> _pasajeros = new List<Pasajeros>();
4     private int _posicion;
5     private string _color; // rojo, verde o comun
6     /*
7      * IMPORTANTE:
8      * ¡¡color puede ser "rojo", "verde" o "comun"!!
9      */
10    public Estacion(int posicion, string color) {
11        _posicion = posicion;
12        _color = color;
13    }
14 }
```

Enumeraciones

Los enums son super útiles para definir atributos categóricos.

```
2 class Estacion {
3     private List<Pasajeros> _pasajeros = new List<Pasajeros>();
4     private int _posicion;
5     private string _color; // rojo, verde o comun
6     /*
7      * IMPORTANTE:
8      * ¡¡color puede ser "rojo", "verde" o "comun"!!
9      */
10    public Estacion(int posicion, string color) {
11        _posicion = posicion;
12        _color = color;
13    }
14 }
```

... definir el color como un string es problemático porque es muy fácil equivocarse.

Enumeraciones

Solución: Crea un enum donde se definan los colores posibles de la estación.

```
2 enum ColorEstacion { roja, verde, comun }
```

Enumeraciones

Solución: Crea un enum donde se definan los colores posibles de la estación.

```
2 enum ColorEstacion { roja, verde, comun }
```

Ahora el color de la estación **debe** ser uno de los elementos en ColorEstacion:

```
2 class Estacion {
3     private List<Pasajeros> _pasajeros = new List<Pasajeros>();
4     private int _posicion;
5     private ColorEstacion _color;
6     public Estacion(int posicion, ColorEstacion color) {
7         _posicion = posicion;
8         _color = color;
9     }
10 }
```

Enumeraciones

Solución: Crea un enum donde se definan los colores posibles de la estación.

```
2 enum ColorEstacion { roja, verde, comun }
```

Ahora el color de la estación **debe** ser uno de los elementos en ColorEstacion:

```
2 class Estacion {
3     private List<Pasajeros> _pasajeros = new List<Pasajeros>();
4     private int _posicion;
5     private ColorEstacion _color;
6     public Estacion(int posicion, ColorEstacion color) {
7         _posicion = posicion;
8         _color = color;
9     }
10 }
```

... y el código para crear una estación queda así:

```
2 Estacion e1 = new Estacion(0, ColorEstacion.comun);
```

Properties

Properties

Dos operaciones comunes son agregar getters y setters de atributos:

```
2 class Person {  
3     private string _name;  
4     public string GetName() // Getter  
5     {  
6         return _name;  
7     }  
8     public void SetName(string name) // Setter  
9     {  
10        _name = name;  
11    }  
12 }
```

Properties

Dos operaciones comunes son agregar getters y setters de atributos:

```
2 class Person {  
3     private string _name;  
4     public string GetName() // Getter  
5     {  
6         return _name;  
7     }  
8     public void SetName(string name) // Setter  
9     {  
10        _name = name;  
11    }  
12 }
```

... y luego usamos estos métodos así:

```
2 Person p = new Person();  
3 p.SetName("Rodrigo");  
4 Console.WriteLine(p.GetName());
```

Properties

C# agregó una sintaxis especial para que crear getters y setters sea más fácil:

```
2 class Person {  
3     private string _name;  
4     public string Name {  
5         get { return _name; } // Getter  
6         set { _name = value; } // Setter  
7     }  
8 }
```

Properties

C# agregó una sintaxis especial para que crear getters y setters sea más fácil:

```
2 class Person {  
3     private string _name;  
4     public string Name {  
5         get { return _name; } // Getter  
6         set { _name = value; } // Setter  
7     }  
8 }
```

... y luego los usamos *como si* el atributo fuera público.

```
2 Person p = new Person();  
3 p.Name = "Rodrigo";  
4 Console.WriteLine(p.Name);
```

Properties

C# agregó una sintaxis especial para que crear getters y setters sea más fácil:

```
2 class Person {  
3     private string _name;  
4     public string Name {  
5         get { return _name; } // Getter  
6         set { _name = value; } // Setter  
7     }  
8 }
```

... y luego los usamos *como si* el atributo fuera público.

```
2 Person p = new Person();  
3 p.Name = "Rodrigo";  
4 Console.WriteLine(p.Name);
```

Observaciones:

- Puedes elegir si crear solo el getter o solo el setter.
- Ambos métodos pueden tener más lógica asociada.

Properties

Esto se puede llevar al siguiente nivel de varias formas.

Properties

Esto se puede llevar al siguiente nivel de varias formas.

1) Si un método consiste en una sola línea se puede utilizar la notación =>.

```
2 class Person {  
3     private string _name;  
4     public string Name {  
5         get => _name; // retorna _name  
6         set => _name = value; // cambia el valor de _name  
7     }  
8 }
```

Properties

Esto se puede llevar al siguiente nivel de varias formas.

2) También podemos dejar que C# defina el atributo privado por debajo.

```
2 public class Person {  
3     public string Name { get; private set; }  
4     public string LastName { get; private set; }  
5     public int Age { get; private set; }  
6     public Person(string name, string lastName, int age) {  
7         Name = name;  
8         LastName = lastName;  
9         Age = age;  
10    }  
11 }
```

Esto solo funciona si get y set no tiene lógica asociada

Tipos Genéricos

Tipos Genéricos

Hasta ahora han estado usando tipos de datos genéricos sin saberlo:

```
2 List<double> lista = new List<double>();  
3 Dictionary<string,int> diccionario = new Dictionary<string,int>();  
4 Image<Rgb24> imagen = new Image<Rgb24>(500, 300);
```

¿En qué consisten?

Tipos Genéricos

Hasta ahora han estado usando tipos de datos genéricos sin saberlo:

```
2 List<double> lista = new List<double>();  
3 Dictionary<string,int> diccionario = new Dictionary<string,int>();  
4 Image<Rgb24> imagen = new Image<Rgb24>(500, 300);
```

¿En qué consisten?

Cuando creas una clase, puede agregar uno o más tipos genéricos en su definición:

```
■ class NombreClase <T1, T2, T3...> {.. }
```

Tipos Genéricos

Hasta ahora han estado usando tipos de datos genéricos sin saberlo:

```
2 List<double> lista = new List<double>();  
3 Dictionary<string,int> diccionario = new Dictionary<string,int>();  
4 Image<Rgb24> imagen = new Image<Rgb24>(500, 300);
```

¿En qué consisten?

Cuando creas una clase, puede agregar uno o más tipos genéricos en su definición:

```
■ class NombreClase <T1, T2, T3...> {.. }
```

Dentro de la clase, puedes usar los tipos T_i como si fuese cualquier otro tipo. Luego, al crear un objeto de la clase, se asigna cada T_i al tipo indicado por el usuario.

Tipos Genéricos

Ejemplo: En Point usamos ints para representar las coordenadas del punto.

```
2 class Point {  
3     private int _x;  
4     private int _y;  
5     public int X { get { return _x; } }  
6     public int Y { get { return _y; } }  
7     public Point(int x, int y) {  
8         _x = x;  
9         _y = y;  
10    }  
11 }
```

Tipos Genéricos

Ejemplo: En Point usamos ints para representar las coordenadas del punto.

```
2 class Point {  
3     private int _x;  
4     private int _y;  
5     public int X { get { return _x; } }  
6     public int Y { get { return _y; } }  
7     public Point(int x, int y) {  
8         _x = x;  
9         _y = y;  
10    }  
11 }
```

... pero usar ints es un poco arbitrario. ¿Por qué no usar float, uint o doubles?

Tipos Genéricos

Usando tipos genéricos podemos dejar que el usuario decida.

```
2 public class Point<CoordType> {  
3     private CoordType _x;  
4     private CoordType _y;  
5     public CoordType X { get { return _x; } }  
6     public CoordType Y { get { return _y; } }  
7     public Point(CoordType x, CoordType y) {  
8         _x = x;  
9         _y = y;  
10    }  
11 }
```

Tipos Genéricos

Usando tipos genéricos podemos dejar que el usuario decida.

```
2 public class Point<CoordType> {  
3     private CoordType _x;  
4     private CoordType _y;  
5     public CoordType X { get { return _x; } }  
6     public CoordType Y { get { return _y; } }  
7     public Point(CoordType x, CoordType y) {  
8         _x = x;  
9         _y = y;  
10    }  
11 }
```

Luego el usuario decide qué tipo de dato usar para representar el punto.

```
2 Point<int> p1 = new Point<int>(3,5);  
3 Point<decimal> p2 = new Point<decimal>(3.42m,5.55m);  
4 int p1X = p1.X;  
5 decimal p2Y = p2.Y;
```


Tipos Genéricos

El único problema es que no podemos hacer mucho con `CoordType` dentro de `Point` porque, a priori, `CoordType` puede ser cualquier cosa.

Tipos Genéricos

Podemos poner restricciones sobre tipos genéricos del tipo “*debe implementar esta interfaz*” usando el comando `where`.

Tipos Genéricos

Podemos poner restricciones sobre tipos genéricos del tipo “*debe implementar esta interfaz*” usando el comando `where`.

```
2 public class Point<CoordType> where CoordType:IComparable {  
3     private CoordType _x;  
4     private CoordType _y;  
5     public CoordType X { get { return _x; } }  
6     public CoordType Y { get { return _y; } }  
7     public Point(CoordType x, CoordType y) {  
8         _x = x; _y = y;  
9     }  
10    public void Max(Point<CoordType> other) {  
11        if (_x.CompareTo(other.X) < 0) _x = other.X;  
12        if (_y.CompareTo(other.Y) < 0) _y = other.Y;  
13    }  
14 }
```

Tipos Genéricos

También podemos utilizar tipos genéricos en métodos:

```
1 public static class Utils
2 {
3     public static void Shuffle<T>(this IList<T> list)
4     {
5         var rnd = new Random();
6         for(var i=list.Count; i > 0; i--)
7             list.Swap(0, rnd.Next(0, i));
8     }
9
10    private static void Swap<T>(this IList<T> list, int i, int j)
11    {
12        (list[i], list[j]) = (list[j], list[i]);
13    }
14 }
```

Excepciones

Excepciones

El manejo de excepciones en C# funciona prácticamente igual que en Python.

Las excepciones **son clases** que representan situaciones anómalas. Y son lanzadas cuando no es posible cumplir con el contrato prometido.

Excepciones

El manejo de excepciones en C# funciona prácticamente igual que en Python.

Las excepciones **son clases** que representan situaciones anómalas. Y son lanzadas cuando no es posible cumplir con el contrato prometido.

```
2 int[] arr = new int[10];
3 for (int i = 0; i < 25; i++) {
4     // Lanza un System.IndexOutOfRangeException
5     // al salirse del rango
6     Console.WriteLine(arr[i]);
7 }
```

```
2 ShowName(null);
3 void ShowName(Person p) {
4     // Lanza un System.NullReferenceException
5     Console.WriteLine(p.Name);
6 }
```

Excepciones

Podemos *atajar* excepciones usando `try/catch/finally`.

Excepciones

Podemos *atajar* excepciones usando try/catch/finally.

```
2  int n = 5;
3  int[] arr = new int[3];
4  try {
5      arr[n] = 0;
6  }
7  catch (IndexOutOfRangeException e) {
8      // Entramos aquí ssi se lanza un "IndexOutOfRangeException"
9      // o una de sus clases derivadas dentro del try{...}
10     Console.WriteLine("Indice fuera de rango!");
11     Console.WriteLine(e.Message); // "e" contiene info sobre el error
12 }
13 finally { // Se ejecuta siempre (se ataje o no una excepción)
14     Console.WriteLine("En el finally");
15 }
```

Pueden existir varios catch (se revisan en orden) y el finally es opcional.

Excepciones

Finalmente, también podemos tirar excepciones:

```
2 public static void Validate(IReadOnlyList<Color> colors)
3 {
4     if (colors.Count != 3) throw new InvalidChrominoException();
5     if (colors.Any(color => color == Color.None)) throw new
6     InvalidChrominoException();
7     ...
8 }
```

Excepciones

Finalmente, también podemos tirar excepciones:

```
2 public static void Validate(IReadOnlyList<Color> colors)
3 {
4     if (colors.Count != 3) throw new InvalidChrominoException();
5     if (colors.Any(color => color == Color.None)) throw new
6     InvalidChrominoException();
7     ...
8 }
```

Y crear nuestras propias excepciones (basta con heredar de alguna excepción existente).

```
2 public class GameEndedException : Exception
3 {
4     public Player Player { get; }
5     public GameEndedException(Player player)
6     {
7         Player = player;
8     }
9 }
```